



15

NEGOTIATE AUTHENTICATION AND OTHER SECURITY PACKAGES



The two previous chapters covered the two main network authentication protocols in Windows, NTLM and Kerberos. However, Windows supports several more packages for performing authentication. In this chapter, we'll briefly cover some of these other security packages.

I'll begin by providing more detail about how applications and security packages can use buffers to pass data back and forth using the SSPI APIs. This will help you understand some of the packages' quirks. Then we'll examine the Negotiate security package, as well as the less common secure channel and CredSSP packages. I'll give a quick overview of some additional configuration options you have when setting up a network authentication context and finish up with a description of what happens when you want to use network authentication inside a process with a lowbox token.

Security Buffers

So far, I've implied that using the SSPI APIs is simple: you generate a client authentication token, pass it to the server application, update the server authentication context, receive a token in response, and repeat the process until the authentication is complete. However, because of the complexity of the supported network authentication protocols, these APIs can accept and return more than just an authentication token.

The authentication context, encryption, and signature APIs accept arrays of generic *security buffer* structures as parameters. This security buffer structure, called `SecBuffer` in the native SDK, is wrapped by the `SecurityBuffer` class in the PowerShell module. Each security buffer structure contains a field that determines what type of data the buffer represents and a sized memory buffer for the contents. You can create a buffer using the `New-LsaSecurityBuffer` PowerShell command, specifying the type and contents of the buffer:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 2, 3)
```

You can specify either a byte array or a string when initializing the data. You also specify a type for the buffer. The following is a short list of the most important buffer types you'll encounter:

Empty Contains no data; sometimes used as a placeholder for a return value

Data Contains initialized data; used to pass and return data, such as a message to encrypt

Token Contains a token; used to pass and return authentication tokens and signatures

PkgParams Contains additional configuration parameters for the security package

StreamHeader Contains the header of a streaming protocol

StreamTrailer Contains the trailer of a streaming protocol

Stream Contains the data of a streaming protocol

Extra Contains extra data generated by the security package

ChannelBindings Contains the channel binding data

You can use security buffers as either input or output, depending on the security package's requirements and the API used. If you want to define an output-only buffer, you can use the `size` parameter when creating the buffer:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Size 1000
```

Sometimes you may want to pass an initialized buffer whose contents the package shouldn't modify. To indicate this, the APIs specify two additional flags you can add to the type:

ReadOnly The buffer is read-only but is not part of the signature.

ReadOnlyWithChecksum The buffer is read-only and should be part of the signature.

You specify these additional flags using the `ReadOnly` OR `ReadOnlyWithChecksum` parameter when creating a buffer, as in the following example:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 2, 3) -ReadOnly
```

Whether the difference between the two read-only flags is honored depends on the security package. For example, NTLM ignores the difference and always adds a read-only buffer to the signature, while Kerberos adds the buffer as part of the signature only if the buffer you supply has the `ReadOnlyWithChecksum` flag.

Using Buffers with an Authentication Context

The SSPI APIs used by the `Update-LsaClientContext` and `Update-LsaServerContext` PowerShell commands take two lists of security buffers: one to use as input to the API and one to use as output. You can specify the list of these buffers using the `InputBuffer` and `OutputBuffer` parameters, as shown in [Listing 15-1](#).

```
❶ PS> $in_buf = New-LsaSecurityBuffer -Type PkgParams -String "AuthParam"
❷ PS> $out_buf = New-LsaSecurityBuffer -Type Data -Size 100
❸ PS> Update-LsaClientContext -Client $client -Token $token -InputBuffer $in_buf
   -OutputBuffer $out_buf
PS> $out_buf.Type
Extra

PS> ConvertFrom-LsaSecurityBuffer $out_buf | Out-HexDump
00 11 22 33
```

Listing 15-1: Using input and output buffers with an authentication context

This listing shows a hypothetical use of input and output buffers during authentication. (You'll see actual examples over the course of this chapter.) This example assumes you've already set up a client authentication context as `$client` and a server authentication token as `$token`.

We first create one input buffer of type `PkgParams` containing a string ❶. The contents of the buffer depend on the package you're using; normally, the API's documentation will tell you what you need to specify. Next, we create an output buffer of type `Data`, allocating a maximum buffer size of 100 bytes ❷. We then update the client context, passing it the server authentication token and the input and output buffers ❸.

The command will add the token as a `Token` type buffer to the start of the input list, and will also append any channel bindings specified when creating the context. Therefore, the input buffer list passed in this case would contain the `Token` buffer followed by the `PkgParams` buffer. Sometimes the package doesn't want you to include the `Token` buffer; in that case, you can specify the `NoToken` parameter to exclude it from the input list.

The command also automatically adds the output `Token` buffer for the new authentication token to the output list. If the API call succeeds, it will assign the contents of this buffer to the context's `Token` property. It's not normally necessary to exclude that buffer from the output, so the command doesn't give you that option.

After a successful call, we check the output buffer, which has been updated. Certain packages might change an output buffer's type, size, and contents. For instance, the type in this example has been changed from `Data` to `Extra`. We can convert the buffer back to a byte array using the `ConvertFrom-LsaSecurityBuffer` command. Displaying the output shows that the 100-byte buffer we've created now has only 4 valid bytes. The security package initialized these 4 bytes and updated the structure's length accordingly.

Using Buffers with Signing and Sealing

Using the `Buffer` parameter, you can specify buffers during signing and sealing operations when calling the `Get-LsaContextSignature` and `Test-LsaContextSignature` PowerShell commands, as well as `Protect-LsaContextMessage` and `Unprotect-LsaContextMessage`. The underlying APIs take only a single list of buffers to use for both the input and output. In [Listing 15-2](#), we encrypt a buffer containing an additional header.

```
PS> $header = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 3, 4)
          -ReadOnlyWithChecksum
PS> $data = New-LsaSecurityBuffer -Type Data -String "HELLO"
PS> $sig = Protect-LsaContextMessage -Context $client -Buffer $header, $data
PS> ConvertFrom-LsaSecurityBuffer -Buffer $header | Out-HexDump
```

```
00 01 03 04

PS> ConvertFrom-LsaSecurityBuffer -Buffer $data | Out-HexDump
D5 05 4F 40 22 5A 9F F9 49 66

PS> Unprotect-LsaContextMessage -Context $server -Buffer $header, $data
-Signature $sig
PS> ConvertFrom-LsaSecurityBuffer -Buffer $data -AsString
HELLO
```

Listing 15-2: Encrypting a message with buffers

We first create the header buffer, marking it as read-only with a checksum. By marking it as read-only, we ensure that the contents won't be encrypted but will still be included in the signature. Next, we create the data buffer from a string.

We then pass the buffers to `Protect-LsaContextMessage`. This command returns the signature for the encryption operation and updates the encrypted data in place. When dumping the buffers, we can see that the header is still unencrypted even though the data buffer has been encrypted.

We can decrypt the buffer using `Unprotect-LsaContextMessage` in a manner similar to how we encrypted the buffer: by passing the buffers and the signature to the command. Once the buffer is decrypted, we can convert it back to a string. If the signature for the buffers isn't valid, the command will throw an error.

Now that you know how to use security buffers for the SSPI APIs, let's look at the Negotiate protocol, which allows Windows to automatically select the best authentication protocol to use based on what credentials are available to the caller.

The Negotiate Protocol

What happens if you don't know what types of network authentication the server supports? You might first try using Kerberos and then, if it isn't supported, switch to NTLM. But that's not a very efficient use of resources. Also, if Microsoft were to later introduce a new, more secure authentication protocol, you'd have to update your application to support it. The *Negotiate* protocol solves both problems by allowing a client and server to negotiate the best available network authentication protocol. Microsoft's implementation of Negotiate is based on the *Simple and Protected Negotiation Mechanism (SPNEGO)* protocol, defined in RFC4178.

To select the Negotiate protocol, use the `Negotiate` package in both the client and the server authentication context. The first token generated by a client authentication context contains a list of the authentication protocols the client supports. In its ASN.1 structure, it can also embed the first authentication token for whichever of the supported authentication protocols the client would prefer to use. For example, it might embed an NTLM NEGOTIATE token. In [Listing 15-3](#), we initialize the Negotiate client authentication context.

```
❶ PS> $credout = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $credout
PS> Format-LsaAuthToken -Token $client.Token
❷ <SPNEGO Init>
❸ Mechanism List      :
    1.3.6.1.4.1.311.2.2.10      - NTLM
    1.2.840.48018.1.2.2        - Microsoft Kerberos
    1.2.840.113554.1.2.2      - Kerberos
    1.3.6.1.4.1.311.2.2.30     - Microsoft Negotiate Extended
❹ <SPNEGO Token>
    <NTLM NEGOTIATE>
    Flags: Unicode, Oem, RequestTarget, Signing, LMKey, NTLM,...
    Domain: MINERAL
    Workstation: GRAPHITE
    Version: 10.0.18362.15
    </SPNEGO Token>
```

Listing 15-3: Initializing the Negotiate client authentication

We specify the credentials for using the Negotiate security package ❶, then continue as normal by creating the context. In the formatted token, we first see `SPNEGO Init`, which indicates that this is an initialization token ❷. Following the header is the list of supported authentication protocols, or *security mechanisms* ❸. The list is sorted in descending order of preference, so in this case, the client prefers NTLM over Kerberos. You won't see Kerberos in the list unless you're on a domain-joined system.

You might notice the mechanism list contains two types of Kerberos. The presence of the `Microsoft Kerberos` identifier is due to a bug in Windows 2000: the value 113554 in the identifier, or 0x1BB92 in hexadecimal, was truncated to 16 bits, resulting in the value 0xBB92, or 48018. Microsoft has left this mistake for backward compatibility reasons, and the two values represent the same Kerberos authentication protocol. Microsoft also defines an extended negotiation protocol, the fourth mechanism in this list, but we won't discuss it here.

Following the list of supported protocols is an authentication token ❹. In this case, the client has chosen to send the initial NTLM NEGOTIATE token.

The server authentication context can select the most appropriate authentication protocol it supports. Most commonly, it will use the protocol that is the client's preferred choice, determined by the ordering of the list of supported authentication protocols. However, it can also ignore the client's preference and request a different authentication protocol if desired. It sends the selected authentication protocol and any further authentication tokens to the client. This authentication exchange process continues until either an error occurs or the process is complete. [Listing 15-4](#) shows how the server responds to the client's request.

```
PS> $credin = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Format-LsaAuthToken -Token $server.Token
<SPNEGO Response>
Supported Mech      : 1.3.6.1.4.1.311.2.2.10 - NTLM
State               : Incomplete
<SPNEGO Token>
<NTLM CHALLENGE>
Flags              : Unicode, RequestTarget, Signing, NTLM, LocalCall, AlwaysSign,...
--snip--
```

Listing 15-4: Continuing the Negotiate authentication on the server

We first pass the client authentication token to the server authentication context that we create. In the formatted output, we can see that it's an SPNEGO Response, and that the server has opted to use NTLM. The response has a State flag, which indicates that the negotiation is currently incomplete. Following that is the authentication token, which, as expected, is now an NTLM CHALLENGE token.

In [Listing 15-5](#), we complete the authentication.

```
PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> Format-LsaAuthToken -Token $client.Token
<SPNEGO Response>
State               : Incomplete
<SPNEGO Token>
❶ <NTLM AUTHENTICATE>
Flags              : Unicode, RequestTarget, Signing, NTLM, LocalCall, AlwaysSign,...
--snip--
```

```
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Format-LsaAuthToken -Token $server.Token
<SPNEGO Response>
❷ State : Completed

❸ PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> $client.PackageName
NTLM
```

Listing 15-5: Completing the Negotiate authentication

The next client authentication token sent is the NTLM AUTHENTICATE token ❶. Note that the supported authentication protocol field is not present. This is only required in the initial server token, and it's omitted from subsequent tokens.

In normal NTLM authentication, the authentication would typically complete at this point. However, in Negotiate authentication, the client's state is considered `Incomplete` until we generate a final server token and update the client with this token, which then marks the state as `Completed` ❷. We can then query the final package using the `PackageName` property ❸, which shows that we negotiated NTLM.

To negotiate the use of Kerberos, the protocol acts in a similar manner. But as Kerberos needs an SPN to function, you must specify the target name using the `Target` parameter when creating the client authentication context; otherwise, the protocol will select NTLM. The output of the Kerberos authentication will replace the NTLM tokens with Kerberos AP-REQ and AP-REP tokens.

Now that we've covered the Negotiate protocol, let's discuss a few less common security packages that you might encounter during an analysis of a Windows system.

Less Common Security Packages

We've covered the three main security packages you're most likely to use on Windows: NTLM, Kerberos, and Negotiate. But there are a few other security packages that have important functions, even if you're less likely to use them directly. We won't spend very much time discussing these, but I'll give you a quick example of each so that you understand their purpose and function.

Secure Channel

Sending sensitive information (like user credentials) unencrypted over the internet is generally considered a bad idea. Several network protocols can encrypt network traffic, but by far the most common is *Transport Layer Security (TLS)*, which was once called *Secure Sockets Layer (SSL)* and was originally developed by Netscape in the mid-1990s to secure HTTP connections. A variant of TLS, the *Datagram Transport Layer Security (DTLS)* protocol, can encrypt traffic from unreliable protocols, such as the *User Datagram Protocol (UDP)*.

Secure channel is an implementation of TLS provided as a security package, and you can access it through the `Schannel` package using the same SSPI APIs as for other network authentication protocols. While you can use secure channel as a TLS or DTLS encryption layer for network traffic, you can also use it to provide client authentication facilities to a server through client certificates.

Let's walk through a simple example of how to use the package. [Listing 15-6](#) starts by setting up the client credentials handle and the client authentication context.

```
PS> $credout = New-LsaCredentialHandle -Package "Schannel" -UseFlag Outbound
PS> $name = "NotReallyReal.com"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $name
           -RequestAttribute ManualCredValidation
PS> Format-LsaAuthToken -Token $client.Token
SChannel Record 0
Type      : Handshake
Version: 3.3
Data      :
           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789ABCDEF
-----
00000000: 01 00 00 AA 03 03 60 35 C2 44 30 A9 CE C7 8B 81 - .....`5.D0.....
00000010: EB 67 EC F3 9A E3 FD 71 05 70 6C BB 92 19 31 C9 - .g.....q.pl...1.
--snip--
```

Listing 15-6: Setting up the secure channel client authentication context

When setting up the context, you need to specify a target name, which is typically the DNS name of the server. The protocol uses this target name to verify that the server has a valid certificate for that name. TLS connections can also be cached, so the protocol can check whether an existing cache entry exists for the target name. In this case, the name won't matter because we specify the `ManualCredValidation` request attribute, which dis-

ables the server certificate checks so that we can use a self-signed certificate for the server.

We then format the authentication token, which displays the TLS protocol's simple record structure (shown in [Figure 15-1](#)).

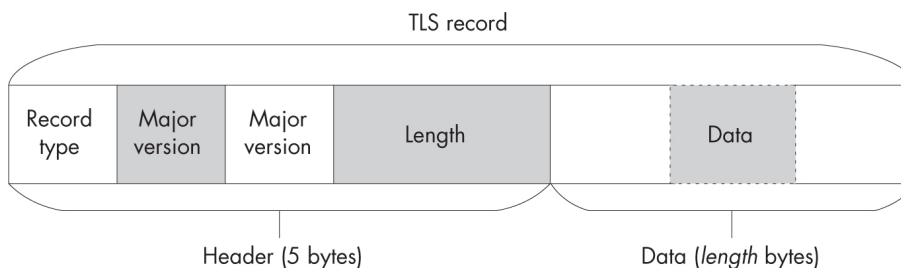


Figure 15-1: The TLS record structure

The record structure contains a 5-byte header consisting of a record type, the major and minor versions of the protocol, and a data length. The header is followed by a list of bytes whose interpretation depends on the record type. In [Listing 15-6](#), the type is `Handshake`, a record used during the connection setup to negotiate the encryption protocol to use, exchange certificates, and communicate the encryption keys. Its version is 3.3, which corresponds to TLS 1.2. (The designers of the protocol considered TLS to be a minor addition to SSL 3.0, so they increased only its minor version number.)

In [Listing 15-7](#), we generate an X.509 certificate and finish setting up the server side of the secure channel authentication.

```

PS> $store = "Cert:\CurrentUser\My"
❶ PS> $cert = Get-ChildItem $store | Where-Object Subject -Match $name
PS> if ($null -eq $cert) {
❷     $cert = New-SelfSignedCertificate -DnsName $name -CertStoreLocation $store
}
❸ PS> $server_cred = Get-LsaSchannelCredential -Certificate $cert
PS> $credin = New-LsaCredentialHandle -Package "Schannel" -UseFlag Inbound
-Credential $server_cred
PS> $server = New-LsaServerContext -CredHandle $credin
❹ PS> while(!(Test-LsaContext $client) -and !(Test-LsaContext $server)) {
    Update-LsaServerContext -Server $server -Client $client
    Update-LsaClientContext -Client $client -Server $server
}

```

Listing 15-7: Initializing a security channel server context and completing authentication

We start by checking whether we have a certificate whose subject name is the DNS name we specified when creating the client authentication context ❶. PowerShell exposes the system's certificate store via the *Cert* drive provider. In this case, we check only the current user's personal certificate store for a matching certificate.

If the certificate doesn't already exist, we create a new one using the `New-SelfSignedCertificate` command with the DNS name as the subject, storing it in the current user's personal store ❷. This certificate isn't trusted for the TLS certificate chain. You could add the new certificate to `Cert:\CurrentUser\Root`, which would make it trusted; however, it's safer to just disable the certificate checking in the client for this example.

To use the certificate for the server, we need to create a set of secure channel credentials, specifying the certificate for use by the server ❸. Note that the certificate must have an associated private key for the server to use. If you pick a certificate without the private key, this line of code will generate an error. We can use the credentials to create a handle and, from that, the server authentication context.

Finally, we exchange tokens between the server and client authentication context until the authentication completes ❹. Of course, in a real application this process would exchange the tokens over a network connection, but for the sake of simplicity, we ignore the network entirely here.

Before we do anything else, we can inspect the negotiated security information, as shown in [Listing 15-8](#).

```
PS> $client.ConnectionInfo
Protocol      Cipher Hash      Exchange
-----
TLS1_2_CLIENT AES_256 SHA_384 ECDH_EPHEM

PS> $client.RemoteCertificate
Thumbprint                                     Subject
-----
2AB144A50D93FE86BA45C4A1F17046459D175176 CN=NotReallyReal.com

PS> $server.ConnectionInfo
Protocol      Cipher Hash      Exchange
-----
TLS1_2_SERVER AES_256 SHA_384 ECDH_EPHEM
```

Listing 15-8: Inspecting the connection information

Note that the `ConnectionInfo` property returns the negotiated protocol and encryption algorithms. In this case, we've negotiated TLS 1.2 using the AES256 encryption algorithm, SHA384 for integrity, and elliptic curve Diffie-Hellman to exchange an ephemeral encryption key.

We can also query the server's certificate. This should match the one we used in the server's credentials. As we specified manual credential validation, we can check whether the certificate is valid; if we hadn't requested manual validation, the handshake process would have generated an error. Finally, we can also query the server's connection information to double-check that it's the same as the client's.

At this point, we've set up the connection, but we have yet to transfer a single byte of user data to the server. [Listing 15-9](#) shows how to encrypt and decrypt application data sent over the network connection.

```

❶ PS> $header = New-LsaSecurityBuffer -Type StreamHeader
   -Size $client.StreamHeaderSize
PS> $data = New-LsaSecurityBuffer -Type Data -Byte 0, 1, 2, 3
PS> $trailer = New-LsaSecurityBuffer -Type StreamTrailer
   -Size $client.StreamTrailerSize
PS> $empty = New-LsaSecurityBuffer -Empty
PS> $bufs = $header, $data, $trailer, $empty
❷ PS> Protect-LsaContextMessage -Context $client -Buffer $bufs -NoSignature
❸ PS> $msg = $header, $data, $trailer | ConvertFrom-LsaSecurityBuffer
PS> $msg_token = Get-LsaAuthToken -Context $client -Token $msg
PS> Format-LsaAuthToken $msg_token
SChannel Record 0
❹ Type      : ApplicationData
Version     : 3.3
Data        :
              00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789ABCDEF
-----
00000000: 00 00 00 00 00 00 00 01 C7 3F 1B B9 3A 5E 40 7E - .....?...^@~
00000010: B0 6C 39 6F EC DA E7 CC CC 33 C2 95 - .19o.....3..

❺ PS> $header = New-LsaSecurityBuffer -Type Data -Byte $msg
PS> $data = New-LsaSecurityBuffer -Empty
PS> $trailer = New-LsaSecurityBuffer -Empty
PS> $empty = New-LsaSecurityBuffer -Empty
PS> $bufs = $header, $data, $trailer, $empty
❻ PS> Unprotect-LsaContextMessage -Context $server -Buffer $bufs -NoSignature
PS> ConvertFrom-LsaSecurityBuffer $data | Out-HexDump
00 01 02 03

```

Listing 15-9: Encrypting and decrypting application data

Secure channel requires passing four buffers to the `Protect-LsaContextMessage` command ❶. The first buffer is for the TLS record header. It needs to be of type `StreamHeader` and should be of a size queried from the context using the `StreamHeaderSize` property.

The second buffer is for the data to encrypt and must be of type `Data`. There is a maximum allowed size for this buffer, which you can query using the `StreamMaxMessageSize` property. The maximum size is typically 16KB, so the 4 bytes we use here should fall well within the limit. If the application data to encrypt is larger than the maximum size, you'll need to fragment the data into smaller parts.

The third buffer will contain the stream trailer, which must be of type `StreamTrailer` and of size `StreamTrailerSize`. The final buffer is an empty one. The secure channel package doesn't seem to use the buffer to store anything, but you must pass it, or the call will fail.

We can now encrypt the data by passing all four buffers to the `Protect-LsaContextMessage` command ❷. One important thing to note is that you should also pass the `NoSignature` parameter. Any generated signature will be part of the generated protocol data, not returned separately, so there is no need for the command to automatically handle the signature.

The result of the encryption is that the header, data, and trailer buffers are populated with the data required to transmit the application data to the server. We need to concatenate the buffers together using the `ConvertFrom-LsaSecurityBuffer` command ❸. In this case, we already know that the data generated is a TLS record, so we can use the authentication context commands to inspect its structure. We can see that the record type is now `ApplicationData` ❹, whereas in [Listing 15-6](#) the record type was `Handshake`. The use of `ApplicationData` indicates that this is an encrypted data record.

Now we need to decrypt the data on the server. To do so, we again need four buffers; however, their configuration is slightly different. For decryption, we must place the entire TLS record in the first buffer as a `Data` type ❺. The next three buffers can be empty; they'll be populated during decryption with the appropriate parts of the message.

We pass the buffers to the `Unprotect-LsaContextMessage` command, again specifying the `NoSignature` parameter, as the signature is part of the protocol ❻. When checking the data buffer, which was originally empty, we now find it's populated with the original unencrypted data.

I've made secure channel look easy to use, but it's much more complex than shown here. For example, you'll have to deal with out-of-band alerts, which indicate problems with the connection. I recommend that you use an existing class (such as `SslStream`, which comes with .NET) to add TLS support to your application unless there's a niche feature not exposed that you need to use.

By default, the TLS protocol verifies only the server in the secure channel connection, using the X.509 certificate; however, the server can request that the client also present a valid certificate for verification purposes. To require the client to send a certificate, specify the `MutualAuth` request attribute when creating the server authentication context. By default, secure channel will try to find a suitable certificate for the user on the client, but you can override this search by setting an explicit certificate when generating the client's credentials.

The server can query for the client's certificate using the same `RemoteCertificate` property on the server authentication context. Note that secure channel doesn't validate the contents of the client certificate by default; doing so is up to the server application. The only thing secure channel guarantees is that the client can prove they have the corresponding private key for the certificate. If the server is part of an enterprise network, it's possible to add an identity certificate to Active Directory so that the client certificate can be mapped to a user account and a `Token` object can be queried for the user's identity without any further authentication.

CredSSP

The final security package we'll look at is *CredSSP*, an authentication protocol developed by Microsoft to improve the security of remote desktop connections to Windows machines. [Figure 15-2](#) shows the original remote desktop implementation.

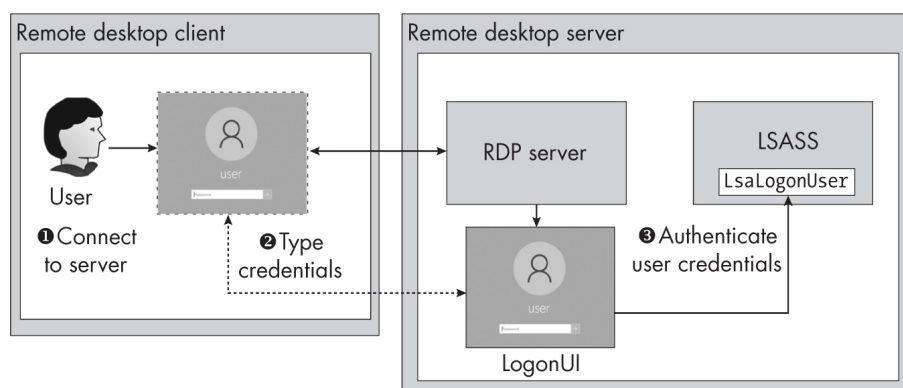


Figure 15-2: The original remote desktop implementation

In the original implementation, a client would connect to the server using a client application ❶. The RDP server would then create a LogonUI for the user that displayed the normal Windows logon user interface and replicate this LogonUI over RDP, so the user would get the same UI on their client machine. The user could then enter their username and password into the LogonUI ❷, which would follow the interactive authentication process outlined in [Chapter 12](#) to verify the user's credentials ❸ and create their desktop.

This approach to implementing a remote desktop has several security problems. First, it performs no verification of the client; this allows anyone to connect, then try to guess a user's password or exploit some bug in the LogonUI to get access to the server. Second, starting up a desktop session for the user interface is quite an expensive operation; it's easy to make enough connections to a remote desktop server to exhaust the machine's resources and cause a denial-of-service condition. Finally, there is a risk of the user having their credentials phished by providing them to a malicious remote server they were tricked into connecting to.

Microsoft's solution to these problems is *Network Level Authentication (NLA)*. NLA is available in Windows Vista onward, and it is the default authentication mechanism used when enabling remote desktop connections. NLA avoids the previously discussed problems by integrating authentication into the Remote Desktop Protocol and verifying that the user has valid credentials before starting a desktop session. This confirms the identity of the client, prevents the expensive operation of setting up the desktop until authentication succeeds, and allows the user to avoid disclosing their credentials to the server.

The CredSSP package implements NLA. It provides TLS for network-level encryption (based on secure channel), and a separate *TS Service Security Package (TSSSP)* that uses the Negotiate protocol to authenticate the user, as well as to derive a session key to encrypt the user's credentials when sending them to the server. [Figure 15-3](#) shows an overview of using NLA to connect to a remote desktop server.

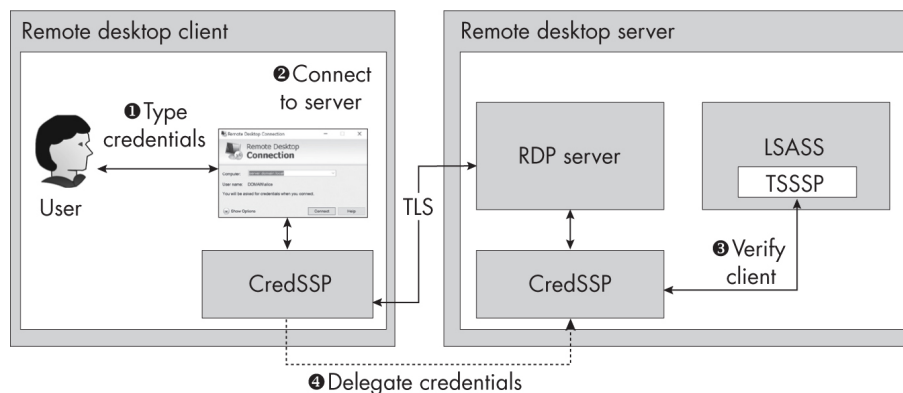


Figure 15-3: A remote desktop connection using Network Level Authentication

First, instead of immediately making a connection, the user provides their credentials to the remote desktop client ❶. This typically consists of their username and password for the remote server.

The client then makes a connection to the remote server, using the CredSSP package to protect the network traffic with TLS ❷. The server sets up a corresponding CredSSP authentication context to implement this communication. Next, its CredSSP context uses the TSSSP package to verify the client based on an existing network authentication protocol, such as NTLM or Kerberos ❸. If this verification step fails, the server can close the connection before creating an expensive desktop.

You might expect the server to create the user's desktop immediately once the network authentication is complete, but there's an additional wrinkle introduced when connecting to a remote desktop. Normally, when you use a network authentication protocol such as NTLM or Kerberos, the created logon session on the server can access only local resources, as the user's credentials are stored on the client computer only. This is the double hop problem I mentioned in [Chapter 13](#) when discussing NTLM domain network authentication.

This behavior is fine if the remote desktop user is accessing a resource locally on the server. But when using a remote desktop, users typically expect to be able to perform single sign-on to other machines on the network to continue to work from that remote desktop session. To solve the single sign-on problem, the client's CredSSP context delegates the user's credentials to the server ❹. It encrypts these credentials using the negotiated session key from the network authentication.

Because the session key for the authentication is derived from the password, a malicious server can't use NTLM relay or forward a Kerberos ticket and then capture the credentials, as they won't be able to decrypt

them. Once the LSA has a copy of the credentials, the remote user can use them to connect to other network services as if they have authenticated interactively.

While CredSSP was designed for use with remote desktop connections, you'll also find it's used for other purposes that require credential delegation. For example, in PowerShell, it's possible to use CredSSP over the WinRM protocol, used for PowerShell remoting. This allows you to create a remote PowerShell session that has the client's credentials and can connect to other systems on the network.

I won't provide an example of using CredSSP, as for the most part it looks like the TLS connection you saw when testing secure channel. Instead, let's cover a few final authentication topics I haven't yet mentioned.

Remote Credential Guard and Restricted Admin Mode

You might notice a problem with delegating your credentials to the remote desktop server. With NLA, you can be confident that the server can verify your credentials, but if an attacker has compromised the server, they could harvest the credentials once they're decrypted during the authentication process. Perhaps an attacker is waiting for you to connect to the server with your privileged domain administrator credentials. Also, there's a chance that the server will leave your credentials lying around in the LSASS process's memory even after you've logged off the system, meaning a malicious attacker can pick them up later.

Windows provides two optional features to mitigate the risk of a compromised server. The first is *Remote Credential Guard*, which works with Kerberos authentication to avoid directly delegating the user's credentials. Using Remote Credential Guard, the client can generate new Kerberos tickets on demand to access resources. This allows the client to connect to other systems from a remote desktop as if they had delegated their credentials.

Importantly for security, this channel to create new tickets exists only while the client is connected to the server. If they disconnect, the server can no longer create new tickets, although any client that is already authenticated will likely stay that way. This means the machine must be actively compromised while the privileged user is authenticated to be useful.

You need to perform some setup steps in your domain to enable Remote Credential Guard. The setup is out of scope for this section, but if the fea-

ture has been enabled, you can use it with the remote desktop client by running the following command line:

```
PS> mstsc.exe /remoteGuard
```

The second security feature is *Restricted Admin mode*. Its big difference from Remote Credential Guard is that when a user authenticates to a server, it creates the logon session without the user's network credentials. Instead, the session is assigned network credentials for the computer account on the server. Therefore, the logon session is primarily useful only if the user wants to perform tasks locally; they won't be able to connect to network resources as themselves unless they explicitly provide their credentials to the remote server. However, this feature ensures that there are no privileged credentials to steal if the server is compromised.

To enable Restricted Admin mode, first add a DWORD registry key value named `DisableRestrictedAdmin` to `HKLM\System\CurrentControlSet\Control\Lsa` and set it to 0. Then you can enable the mode when executing the client with the following command line:

```
PS> Mstsc.exe /RestrictedAdmin
```

One advantage of these two security features (above and beyond the restrictions they place on credential delegation) is that they allow the remote desktop client to use single sign-on authentication based on the current user's credentials stored in the LSA logon session. This is because neither feature requires the plaintext credentials.

The Credential Manager

One annoyance of using a remote desktop connection is having to enter your password every time you want to connect. This seems unavoidable, as you must provide the account password to the server to allow single sign-on to function from the remote desktop server. However, the LSA supports a feature to save the account password for subsequent authentication to save you typing it in again. One place where this feature is used is when you type in your credentials; you'll see a "Remember me" checkbox in the dialog, as shown in [Figure 15-4](#).

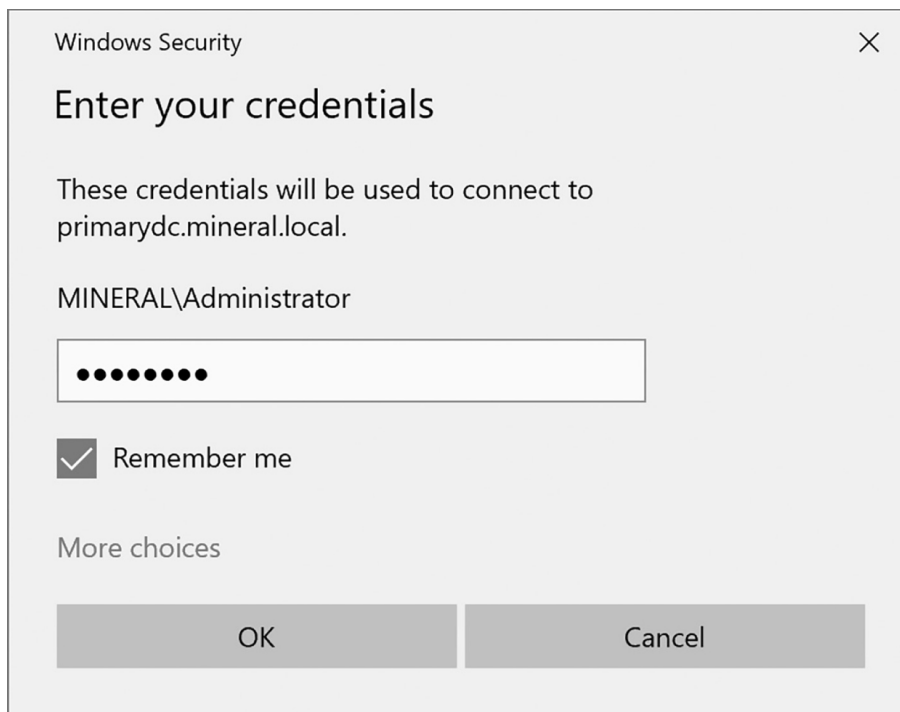


Figure 15-4: Entering and saving your credentials

If you check the box and successfully authenticate, the dialog in which to enter the server's name should change slightly the next time you open it ([Figure 15-5](#)).

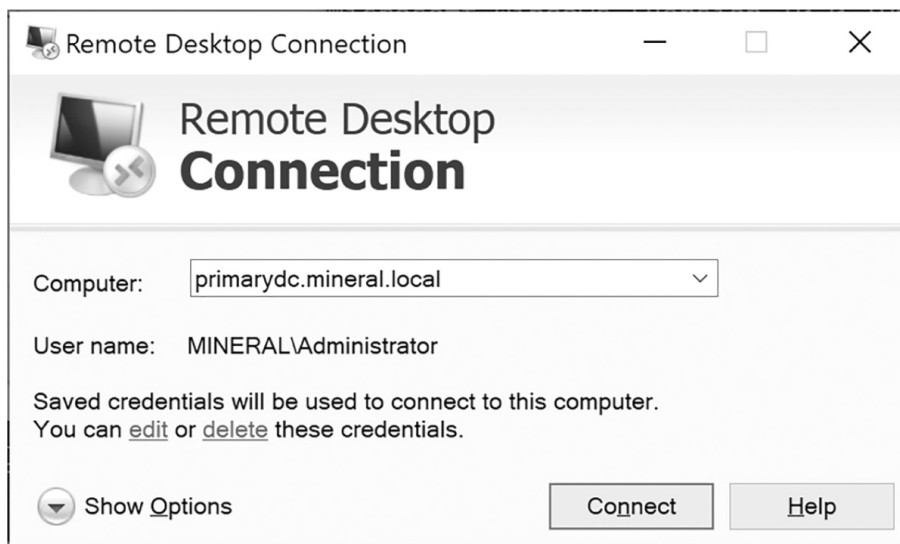


Figure 15-5: Connection dialog with saved credentials

Now you can see that the dialog gives you the option to edit or delete saved credentials for this server.

It would be easy for the client to store the user's password directly to a file on disk, but that wouldn't be very secure. Instead, it uses a service provided by the LSA known as the *credential manager*. The service can store domain passwords for easy reuse, although Microsoft doesn't rec-

ommend this practice. To demonstrate how credentials get stored, [Listing 15-10](#) first uses the `Get-Win32Credential` PowerShell command, which calls the `CredRead` Win32 API, to read the credentials for the remote desktop client.

```
PS> Get-Win32Credential "TERMSRV/primarydc.domain.local" DomainPassword |
Format-Table UserName, Password
UserName      Password
-----
MINERAL\Administrator
```

Listing 15-10: Getting the credentials for a remote desktop client

The credentials are stored by target name, which for domain credentials is the SPN for the service (in this case, `TERMSRV/primarydc.domain.local`). When looking up credentials you also need to specify the type, which in this case is `DomainPassword`.

Here, we've formatted the output to show only the username and password. However, you might notice a problem: the password column is empty. This is an intentional behavior of the service. If the credentials represent a domain password, the password won't be returned unless the caller is running within the LSA process.

This behavior is fine for its intended purpose: to use in security packages that are running inside the LSA. For example, `CredSSP` can check whether the user has a credential for the target remote desktop service based on its SPN and use it to read the user's password to automatically authenticate. The service stores the credentials in individual files in the user's profile, as illustrated in [Listing 15-11](#).

```
PS> ls "$env:LOCALAPPDATA\Microsoft\Credentials" -Hidden
Directory: C:\Users\alice\AppData\Local\Microsoft\Credentials
Mode                LastWriteTime         Length Name
----                -
-a-hs-    5/17      10:15 PM         4076 806C9533269FB8C19A759596441A2ECF
-a-hs-    5/17       9:49 PM          420 B5E4F2A09B2613B8305BA6A43DC15D1F
-a-hs-    5/6       6:33 PM        11396 DFBE70A7E5CC19A398EBF1B96859CE5D
-a-hs-    5/17       3:56 PM         1124 E05DBE15D38053457F3523A375594044
```

Listing 15-11: Viewing the user's credential files

Each file is encrypted using a per-user key through the Data Protection API (DPAPI), which I mentioned in [Chapter 10](#). This means we should be

able to decrypt our own credential files using the DPAPI, through the .NET ProtectedData class. [Listing 15-12](#) enumerates the current user's credential files and tries to decrypt each one using ProtectedData.

```
PS> Add-Type -AssemblyName "System.Security"
PS> ls "$env:LOCALAPPDATA\Microsoft\Credentials" -h | ForEach-Object {
    $ba = Get-Content -Path $_.FullName -Encoding Byte
    [Security.Cryptography.ProtectedData]::Unprotect($ba,$null,"CurrentUser")
}
Exception calling "Unprotect" with "3" argument(s): "The data is invalid."
--snip--
```

Listing 15-12: Attempting to decrypt the user's credential files

Unfortunately, every file returns the same error: The data is invalid. While it is encrypted using the user's DPAPI key, the LSA sets a special flag in the binary data that indicates that only code running in the LSA can decrypt it.

There are many ways to decrypt the files successfully: for example, you could inject code into the LSA process and decrypt them from there, or you could derive the DPAPI key using the user's password and the values from the SECURITY database registry key and decrypt them yourself. If you want to go down the latter route, I'd suggest checking out existing tooling such as Mimikatz, which already implements this functionality.

Another approach to decrypting the files was introduced in Windows Vista. A special token privilege, `SeTrustedCredmanAccessPrivilege`, allows a process to be considered trusted by the LSA when accessing select credential manager APIs. The most interesting of these select APIs is `CredBackupCredentials`, which will back up all of a user's credentials into a file that can later be used to restore the credentials if needed. The backup also contains any protected password values.

[Listing 15-13](#) shows how to back up a user's credentials from the credential manager. You must run these commands as an administrator, as you need to access a privileged process to get a token with `SeTrustedCredmanAccessPrivilege`, which is only granted to select process types.

```
PS> Enable-NtTokenPrivilege SeDebugPrivilege
❶ PS> $token = Use-NtObject($ps = Get-NtProcess -Name "winlogon.exe"
    -Access QueryLimitedInformation) {
    $p = $ps | Select-Object -First 1
```

```

    Get-NtToken -Process $p -Duplicate
}
❷ PS> $user_token = Get-NtToken
PS> $ba = Invoke-NtToken -Token $token {
    ❸ Enable-NtTokenPrivilege SeTrustedCredmanAccessPrivilege
    Backup-Win32Credential -Token $user_token
}
❹ PS> Select-BinaryString -Byte $ba -Type Unicode |
    Select-String "^Domain:" -Context 0, 2
> Domain:target=TERMSRV/primarydc.mineral.local
    MINERAL\Administrator
    Passw0rd10

```

Listing 15-13: Backing up a user's credentials from the credential manager

We first open the privileged Winlogon process and take a copy of its primary token ❶. Next, we get a copy of the user token we want to back up, which in this case is the current process token ❷. We can then impersonate the token we duplicated from Winlogon ❸, enable

SeTrustedCredmanAccessPrivilege, and call the Backup-Win32Credential PowerShell command, which calls the underlying CredBackupCredentials API.

The command returns a byte array containing the backup. The byte array is in a proprietary format, so we select all its Unicode strings and look for any that start with the string Domain: ❹. We can see the stored remote desktop service credentials, including the name and password.

The credential manager is a better place than a user-accessible file to store credentials for use by LSA security packages such as NTLM, Kerberos, and CredSSP. However, that doesn't mean you should use it. While disclosing the credentials takes some work, like any protection mechanism, it must at some point provide the unencrypted values, which an attacker can then extract.

Additional Request Attribute Flags

When you create a client or server authentication context, you can specify a set of request attribute flags to change the behavior of the authentication. We've already seen support for signing and sealing, as well as delegation and mutual authentication, in the previous chapters. Still, it's worth highlighting a few other flags that Kerberos and NTLM support.

Anonymous Sessions

What if you don't know a user account on the target server? SSPI supports the concept of an *anonymous session*, also referred to as a *NULL session*. In an anonymous session, the authenticating user doesn't need any credentials to generate the authentication tokens. The server will process the authentication as usual, but it will generate a token for the *ANONYMOUS LOGON* user. This allows a network protocol to always require authentication, simplifying the protocol, and to then enforce access based on the identity of the authenticated user. You can specify an anonymous session by using the `NullSession` request attribute flag when creating the client authentication context, as in [Listing 15-14](#).

```
PS> $client = New-LsaClientContext -CredHandle $credout  
-RequestAttribute NullSession
```

Listing 15-14: Adding the NullSession request attribute flag

If you then perform local NTLM network authentication, you should notice a change in the `NTLM AUTHENTICATE` token, shown in [Listing 15-15](#).

```
<NTLM AUTHENTICATE>  
❶ Flags      : Unicode, RequestTarget, NTLM, Anonymous,...  
  Workstation: GRAPHITE  
❷ LM Response: 00  
  NT Response:  
    Version   : 10.0.18362.15  
    MIC       : 3780F9F6EC815DD34BA8A643162DC5FC  
  
PS> Format-NtToken -Token $token  
❸ NT AUTHORITY\ANONYMOUS LOGON
```

Listing 15-15: The NTLM AUTHENTICATE token in an anonymous session

The `NTLM AUTHENTICATE` token has the `Anonymous` flag set ❶. Also, the LM response is a single zero byte, and the NT response is missing ❷. Querying the process's `Token` object shows that it's the anonymous user's ❸.

In Kerberos, the anonymous authentication token looks like that for NTLM, as shown in [Listing 15-16](#).

```
<KerberosV5 KRB_AP_REQ>  
Options      : None  
<Ticket>
```

```

Ticket Version      : 0
ServerName          : UNKNOWN -
Realm               :
Encryption Type     : NULL
Key Version         : 0
Cipher Text         :
00000000: 00
<Authenticator>
Encryption Type     : NULL
Key Version         : 0
Cipher Text         :
00000000: 00

```

Listing 15-16: Sending an anonymous Kerberos AP-REQ message

The client sends an AP-REQ message with a ticket and authenticator containing empty values. If you see this message in a network capture, you can be certain the client is establishing an anonymous session.

Identity Tokens

When you perform a network authentication, the final `Token` object is an Impersonation-level token. If the server can pass the impersonation checks described in [Chapter 4](#), it can now access that user's resources. What if we don't want the server to be able to use our identity to access resources? In this case, we can specify the `Identify` request attribute flag, as shown in [Listing 15-17](#), to allow the server to receive only an Identification-level impersonation token, rather than a full Impersonation-level token.

```
PS> $client = New-LsaClientContext -CredHandle $credout -RequestAttribute Identify
```

Listing 15-17: Adding the Identify request attribute flag

This will prevent the server from using our identity to access resources, but still allow it to check who has authenticated. If we then rerun the authentication, we should notice a change in the NTLM `AUTHENTICATE` token, as shown in [Listing 15-18](#).

```

<NTLM AUTHENTICATE>
❶ Flags          : Unicode, RequestTarget, NTLM, Identity,...
--snip--

PS> Format-NtToken -Token $token -Information
TOKEN INFORMATION

```

```

-----
Type      : Impersonation
② Imp Level : Identification

```

Listing 15-18: Examining the flags in the NTLM AUTHENTICATE token and displaying the created token's impersonation level

The NTLM AUTHENTICATE token's flags now include an Identity flag ①. This indicates to the server that the client wants to allow the use of an Identification-level token only. When we get the token from the server authentication context and format it, we can see that the impersonation level is indeed set to Identification ②.

As with NullSession, the Identify request attribute flag will work with Kerberos as well. [Listing 15-19](#) shows that specifying this flag results in an Identity flag being set in the AP-REQ authenticator's GSSAPI Checksum field.

```

<Authenticator>
--snip--
Checksum      : GSSAPI
Channel Binding : 00000000000000000000000000000000
Context Flags  : Identity

```

Listing 15-19: The Identity flag in an AP-REQ GSSAPI checksum

Network Authentication with a Lowbox Token

When a process is running with a lowbox token (described in [Chapter 4](#)), the LSA enforces restrictions on the use of network authentication. This is to make it harder for the sandbox application to abuse network authentication to get access to the user's logon session credentials and, through them, access their resources.

If the lowbox process can create a client authentication context, however, it can generate authentication tokens in only the following three scenarios:

- Using logon session credentials with the Enterprise Authentication capability
- Using logon session credentials to a known web proxy
- Using explicit credentials, such as a username and password

Let's discuss each of these scenarios.

Authentication with the Enterprise Authentication Capability

The *enterprise authentication capability*, represented by the SID `S-1-15-3-8`, can be granted when a lowbox token is created. With this capability, the lowbox process can use the user's logon session credentials to generate any supported network authentication tokens, such as those for NTLM or Kerberos, without restriction.

The enterprise authentication capability is designed for enterprises to use in their internal applications. Outside of enterprises, the primary means of deploying lowbox processes is via the Microsoft App Store, which has restricted the use of this capability in the application submission guidelines. If you apply to the Microsoft store with an application that uses the enterprise authentication capability, it must pass an extra review and might be rejected. However, if you're creating the lowbox token outside of a store application for testing purposes, there is no restriction, as demonstrated in [Listing 15-20](#).

```
PS> $cred = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
PS> $sid = Get-NtSid -PackageName "network_auth_test"
❶ PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Invoke-NtToken $token {New-LsaClientContext -CredHandle $cred}
}
❷ Exception calling ".ctor" with "5" argument(s): "(0x80090304) - The Local
Security Authority cannot be contacted"

PS> $cap = Get-NtSid -KnownSid CapabilityEnterpriseAuthentication
❸ PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid
-CapabilitySid $cap) {
    ❹ $auth = Invoke-NtToken $token {New-LsaClientContext -CredHandle $cred}
    Format-LsaAuthToken $auth
}
<SPNEGO Init>
Mechanism List      :
1.3.6.1.4.1.311.2.2.10      - NTLM
1.2.840.48018.1.2.2        - Microsoft Kerberos
--snip--
```

Listing 15-20: Testing the lowbox enterprise authentication capability

We first create a lowbox `Token` object without the capability ❶. When we create the client authentication context using `New-LsaClientContext`, we get an error ❷. This error comes from the `InitializeSecurityContext` API, which PowerShell calls behind the scenes. Next, we create the lowbox token

with the capability ③. This time, we can successfully create a client authentication context and format the client authentication token ④.

Authentication to a Known Web Proxy

The lowbox process can generate tokens for authentication to web proxies, which commonly require that a domain user can access the internet. To support this use case, you can perform network authentication with the user's logon session credentials if the target name is set to the address of an approved proxy server.

For example, say the target name is HTTP/proxy.mineral.local. The system administrator must configure the proxy address either through the group policy or by using a *Proxy Auto-Configuration (PAC)* script, which makes sure that a web request with an arbitrary proxy configuration won't pass the LSA's checks. [Listing 15-21](#) demonstrates the use of a web proxy target name to allow network authentication. You must have configured a system web proxy for this script to work.

```
PS> $cred = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
① PS> $client = New-Object System.Net.WebClient
PS> $proxy = $client.Proxy.GetProxy("http://www.microsoft.com").Authority
② PS> $target = "HTTP/$proxy"
PS> $target | Write-Output
HTTP/192.168.0.10:1234

PS> $sid = Get-NtSid -PackageName "network_auth_test"
③ PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    ④ $client = Invoke-NtToken $token {
        New-LsaClientContext -CredHandle $cred -Target $target
    }
    Format-LsaAuthToken $client
}
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign,...
```

Listing 15-21: Testing lowbox web proxy authentication

First, we query for the proxy setting using the `WebClient` .NET class ①. We then build the target SPN with an HTTP service class and the proxy address ②.

Next, we create the lowbox token ③. Notice that we haven't specified the enterprise authentication capability. We create the client authentication

context and use the target SPN ❹. The initial authentication succeeds, and we can perform the client authentication to the target proxy.

This proxy authentication is considered secure because the service should check the target name before permitting the authentication. If the lowbox process generates the authentication for the proxy SPN but then sends it to an SMB server, the authentication process should fail. For Kerberos authentication, the SPN selects the key to use for the ticket, so an incorrect SPN should make the ticket fail to decrypt if sent to the wrong service.

Authentication with Explicit Credentials

The final option, shown in [Listing 15-22](#), is to specify explicit credentials when creating the credentials handle provided to the client authentication context.

```
PS> $cred = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
-ReadCredential
UserName: user
Domain: GRAPHITE
Password: *****

PS> $sid = Get-NtSid -PackageName "network_auth_test"
PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Invoke-NtToken $token {
        ❶ $c = New-LsaClientContext -CredHandle $cred -Target "CIFS/localhost"
        Format-LsaAuthToken $c
    }
}
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign,...
```

Listing 15-22: Initializing the client authentication context with explicit credentials

To initialize the client authentication context, you still need to provide a target SPN ❶. However, you don't need to specify a known proxy, as the target can be any service or host. In this case, we specify the CIFS/ localhost SPN.

When in a lowbox token sandbox, you can act as a server for network authentication, as it's possible to get a Token object for a different user. However, unless the token's user exactly matches the caller's user and lowbox package SID, the returned token is set to the Identification level,

which prevents it from being abused to elevate privileges. The restriction on the impersonation level applies even if the lowbox token has the enterprise authentication capability, as this grants access to the client authentication context only.

BYPASSING THE PROXY CHECK

Microsoft very poorly documents these bypasses of the capability requirement for proxy authentication. The problem with security features for which there is little to no official documentation is that few developers know they exist, so they don't get tested as rigorously as they should, especially for unusual edge cases. In a utopian world, Microsoft would have implemented comprehensive security tests for the proxy check feature, but sadly, we don't live in such a world.

While researching the proxy check for this book, I reverse engineered its implementation in the LSA and noticed that if the target name isn't a proxy, the authentication process continues, but the LSA sets a flag for the security package that indicates it must use explicitly provided credentials. As we saw when we covered NTLM in [Chapter 13](#), it's possible to provide the username and domain for the current user but leave the password empty; in that case, the security package will use the password from the logon credentials.

If you specify just the username and domain, the NTLM security package will consider them to be explicit credentials, satisfying the flag set by the LSA even though the authentication will use default credentials. This bypasses all the checks and grants a lowbox process access to the default user, which an attacker could abuse to access network resources accessible by that user. You can learn more about this issue in CVE-2020-1509.

Even after Microsoft implemented a fix, I was still able to bypass the check, as during my research I also noticed that the check for the target name wasn't implemented correctly. Recall from [Chapter 13](#) that a target name is an SPN composed of three parts, separated by forward slashes: the service class, the instance name, and the service name. The parsing and checking code in the LSA had two problems:

- It didn't verify that the service class was HTTP or HTTPS.
- It checked the service name for the proxy address, not the instance name.

Not verifying the service class allowed the target name to refer to other services, such as CIFS, to use for authenticating to an SMB server. This let me

construct a target name of the form

CIFS/fileserver.domain.com/proxy.domain.com. If proxy.domain.com was a registered proxy, this target name would pass the proxy check; however, the SMB server would care only about the service class and the instance name (here, fileserver.domain.com), and once again would allow access to the user's default credentials. Microsoft fixed this issue as well, although without assigning it a CVE number.

The main root cause of the service name problem was that the API Microsoft used to parse the SPN would set the service name component to match the instance name component if no service name were provided. For example, HTTP/proxy.domain.com would set both the instance name and the service name to proxy.domain.com. Therefore, this code worked in Microsoft's limited testing, but broke when someone decided to test the feature's edge cases. I mentioned the target name parsing bypass to Microsoft when reporting the original issue, but for some reason, it wasn't fixed at the same time. In addition to supporting my previous statement about undocumented features often not being very well tested, this example demonstrates why you should always verify any changes a developer makes to ensure they've implemented a comprehensive fix.

That said, Microsoft recommends disabling automatic authentication to HTTP proxy servers when it's not required by adding the `AllowUnprivilegedProxyAuth` registry key value to

`HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Lsa` and setting its value to 0. If the value doesn't exist, Windows enables this authentication by default if targeting a proxy.

The Authentication Audit Event Log

Let's wrap up our discussion of authentication with an overview of the auditing data generated during interactive and network authentication. When you're monitoring an enterprise network, you might want to know which users have attempted to authenticate to the Windows system. By analyzing the audit log, you can identify their successful and unsuccessful authentication attempts to a machine.

You can find the authentication audit log records in the same security event log we inspected in [Chapter 9](#) when discussing object audit events. We can use a similar technique of filtering the log by event ID to get the events we're interested in. Here are some event IDs for important authentication events:

4624 An account logged on successfully.

4625 An account failed to log on.

4634 An account logged off.

Let's look at the information these events provide. [Listing 15-23](#) starts by querying the security event log for the successful logon event, 4624. Run this command as an administrator.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4624)} |
Select-Object -ExpandProperty Message
An account was successfully logged on.

Subject:
    Security ID:          S-1-5-18
    Account Name:         GRAPHITE$
    Account Domain:       MINERAL
    Logon ID:             0x3E7

Logon Information:
    Logon Type:           2
    Restricted Admin Mode: No
    Virtual Account:      No
    Elevated Token:       Yes

Impersonation Level:      Impersonation

New Logon:
    Security ID:          S-1-5-21-1195776225-522706947-2538775957-1110
    Account Name:         alice
    Account Domain:       MINERAL
    Logon ID:             0x15CB183
    Linked Logon ID:      0x15CB1B6
    Network Account Name:  -
    Network Account Domain: -
    Logon GUID:           {d406e311-85e0-3932-dff5-99bf5d834535}

Process Information:
    Process ID:           0x630
    Process Name:         C:\Windows\System32\winlogon.exe

Network Information:
    Workstation Name:     GRAPHITE
    Source Network Address: 127.0.0.1
    Source Port:          0

Detailed Authentication Information:
    Logon Process:        User32
```

Authentication Package:	Negotiate
Transited Services:	-
Package Name (NTLM only):	-
Key Length:	0

Listing 15-23: A log record for a successful interactive authentication event

This listing shows an example entry for a successful authentication event. On a frequently used system there are likely to be many such entries, so pick just one to inspect.

The event records contain a lot of information, some of which might not be populated for certain logon types. Each entry starts with information about the user account that has made the authentication request. For an interactive authentication, you'll likely find this to be a privileged account, such as the *SYSTEM* computer account. Next comes information about the logon, including the logon type. The 2 indicates interactive. Some other logon types are network (3), batch (4), service (5), and remote interactive (10). This section also indicates whether Restricted Admin mode was used for the authentication and whether the session the event represents is elevated. It's followed by an indication of the token's impersonation level.

The following section contains the details of the logon session created for the successful authentication, including the user's SID, name, and domain. As this is an elevated interactive authentication, we see two logon IDs: one for the session itself and one for the linked, non-elevated logon session created for UAC.

Next come the details of the process making the authentication request. In this example, it's the process that called `LsaLogonUser`. The final two sections contain network authentication information and additional details that didn't fit into other categories. Part of the detailed authentication information is the security package used for the authentication. In this case, Negotiate was used, so it will have chosen the best authentication protocol for the user.

You'll see the same type of event record generated regardless of whether authentication occurred through `LsaLogonUser` or through network authentication. For example, if the event is for an NTLM network authentication, you should see something like [Listing 15-24](#) in the detailed authentication information section.

Detailed Authentication Information:

Logon Process:	NtLmSsp
Authentication Package:	NTLM
Transited Services:	-
Package Name (NTLM only):	NTLM V2
Key Length:	128

Listing 15-24: The detailed information for a successful NTLM network authentication

Let's now look at a failed authentication event. [Listing 15-25](#) queries for events with an ID of 4625, as an administrator.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4625)} |
Select-Object -ExpandProperty Message
An account failed to log on.
--snip--
Account For Which Logon Failed:
    Security ID:      S-1-0-0
    Account Name:     alice
    Account Domain:   MINERAL

Failure Information:
    Failure Reason:   Unknown user name or bad password.
    Status:           0xC000006D
    Sub Status:       0xC000006A
--snip--
```

Listing 15-25: A failed authentication event log record

In the output, I've highlighted just one record. It has many of the same sections as for a successful authentication, so I've removed anything that appears in both types of record.

The first of the sections shown here contains details on the user account that failed to authenticate. The SID entry isn't guaranteed to be valid; for example, in this case, the SID does not represent the *alice* user. Next, we get more details about the failure, starting with a text version of the error, followed by the status, which here is an NT status code of `STATUS_LOGON_FAILURE`. The sub-status code provides more detail; in this case, it's `STATUS_WRONG_PASSWORD`, which indicates that the user did not provide a valid password. Other sub-status codes you might encounter include `STATUS_NO_SUCH_USER`, if the user doesn't exist, and `STATUS_ACCOUNT_DISABLED`, if the user's account has been disabled.

Finally, we'll look at a log-off event, generated when a logon session is deleted. This typically occurs when no `Token` objects that reference the logon session remain. Run the command in [Listing 15-26](#) as an administrator.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4634)} |  
Select-Object -ExpandProperty Message  
An account was logged off.  
  
Subject:  
    Security ID:      S-1-5-21-1195776225-522706947-2538775957-1110  
    Account Name:    alice  
    Account Domain:  MINERAL  
    Logon ID:        0x15CB183  
  
Logon Type:          2
```

Listing 15-26: A log-off authentication event log record

This event log record is much simpler than those for successful or failed authentication. It contains just the subject information, including the username and domain. To match a successful authentication event to the corresponding log-off event, you can compare the logon IDs.

Worked Examples

Let's finish with some worked examples using the commands you've learned about in this chapter.

Identifying the Reason for an Authentication Failure

I noted in the previous section that you'll see two status codes in the event log when an authentication process fails: there's the main status, typically `STATUS_LOGON_FAILURE`, and a sub-status, such as `STATUS_WRONG_PASSWORD`. Unfortunately, the event log automatically converts only the main status code to a string, then typically generates a generic "The username or password is incorrect" message that isn't very helpful in diagnosing authentication failures. Let's write a quick script to analyze the event log records and convert the sub-status codes to messages automatically.

One immediate problem we must solve is how to get the sub-status code from the event log record. You could try to manually parse it from the text message. However, you'll see different messages for different languages, and you might not be able to rely on the presence of a text string

such as `SubStatus`. The event log record, however, does contain all its important information as separate properties, and you can query for these using the `Properties` property on the event log record object. [Listing 15-27](#) shows the output generated by such a query.

```
PS> $record = Get-WinEvent -FilterHashtable @{logname='Security';id=@(4634)} | Select
PS> $record.Properties
Value
-----
S-1-5-21-1195776225-522706947-2538775957-1110
alice
MINERAL
--snip--
```

Listing 15-27: Displaying an event log's record properties

Unfortunately, the list of properties contains only the values, with no indication of the properties' names. We want the property with the name `SubStatus`, which might always be at the same index in the properties list, but there is no guarantee that will always be the case. So, to get this information we must manually inspect the XML that stores the event log's properties. We can request this by using the `ToXml` method on the record. [Listing 15-28](#) shows how to extract named properties from an event log record.

```
PS> function Get-EventLogProperty {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, Position = 0, ValueFromPipeline)]
        [System.Diagnostics.Eventing.Reader.EventRecord]$Record
    )

    PROCESS {
        ❶ $xml = [xml]$Record.ToXml()
        $ht = @{
            TimeCreated = $Record.TimeCreated
            Id = $Record.Id
        }
        ❷ foreach($ent in $xml.Event.EventData.data) {
            $ht.Add($ent.Name, $ent."#text")
        }
        [PSCustomObject]$ht
    }
}
PS> Get-EventLogProperty $record
SubjectUserName      : alice
```

```

TimeCreated           : 2/24 1:15:06 PM
IpPort                : -
SubjectLogonId        : 0x54541
KeyLength             : 0
LogonProcessName      : Advapi
IpAddress             : -
LmPackageName         : -
TransmittedServices   : -
WorkstationName       : GRAPHITE
SubjectUserSid        : S-1-5-21-1195776225-522706947-2538775957-1110
❶ SubStatus           : 0xc000006a
AuthenticationPackageName : Negotiate
SubjectDomainName     : MINERAL
ProcessName           : C:\Program Files\PowerShell\7\powershell.exe
❷ FailureReason       : %%2313
LogonType              : 3
Id                    : 4625
Status                : 0xc000006d
TargetUserSid         : S-1-0-0
TargetDomainName      : mineral.local
ProcessId             : 0xe48
TargetUserName        : alice

```

Listing 15-28: Extracting the named event log record properties

We start by defining the `Get-EventLogProperty` function, which will convert each record to a new object. We need to extract an event log record's XML and then parse it into an XML document ❶. The `EventData` XML element stores the properties, so we use the object model PowerShell provides to extract each element and build a hash table from the property name and body text ❷. We then convert the hash table to a custom PowerShell object to make it easier to query.

When inspecting the new object's properties, we find that the `SubStatus` property is now easily accessible ❸. There are some limitations with our approach; for example, we haven't converted the failure reason from a resource identifier to a string ❹. However, we don't need the failure reason, as we can get the message from the status code if we want it.

Now let's expand our code to extract the sub-status for authentication failures ([Listing 15-29](#)).

```

❶ PS> function Get-AuthFailureStatus {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, Position = 0, ValueFromPipeline)]

```

```

    $Record
)

PROCESS {
    [PSCustomObject]@{
        TimeCreated = $Record.TimeCreated
        UserName = $Record.TargetUserName
        DomainName = $Record.TargetDomainName
        ❷ SubStatus = (Get-NtStatus -Status $Record.SubStatus).StatusName
    }
}
}

❸ PS> Get-NtToken -Logon -User $env:USERNAME -Domain $env:USERDOMAIN
-Password "InvalidPassword"
PS> Get-NtToken -Logon -User "NotARealUser" -Domain $env:USERDOMAIN -Password "pwd"
❹ PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4625)} |
Select-Object -First 2 | Get-EventLogProperty | Get-AuthFailureStatus
TimeCreated      UserName      DomainName      SubStatus
-----
2/24      1:15:06 PM  alice      MINERAL      STATUS_WRONG_PASSWORD
2/24/      1:14:45 PM  NotARealUser  MINERAL      STATUS_NO_SUCH_USER

```

Listing 15-29: Parsing authentication failure properties and converting their sub-status codes

We start by defining a function that converts the record properties into a simpler authentication failure object ❶. We pull out only the timestamp, the username, and the domain name, and then convert the `SubStatus` property to its NT status name ❷.

We then perform two failed authentications to generate some entries in the event log ❸. We filter the log to return only authentication failure records, then convert the records in the pipeline ❹. In the generated output, we can see two entries. The first has `STATUS_WRONG_PASSWORD` as the sub-status, indicating that the user was valid but the password was not. The second has `STATUS_NO_SUCH_USER`, which indicates that the user doesn't exist.

Using a Secure Channel to Extract a Server's TLS Certificate

Next, let's walk through a simple example of how to use the secure channel authentication protocol. We'll make a TCP connection to a secure web server and extract its server certificate, then use it to retrieve details about the organization that might own the server and whether the certificate is valid.

Note that there are likely much better ways of getting the server's certificate than the approach taken in this example. For example, most web browsers will allow you to display and export the certificate by browsing to the server. However, that wouldn't help you learn much about how secure channel works. To get started, copy the contents of [Listing 15-30](#) into the script file *get_server_cert.ps1*.

```

❶ param(
    [Parameter(Mandatory, Position = 0)]
    [string]$Hostname,
    [int]$Port = 443
)

$ErrorActionPreference = "Stop"

❷ function Get-SocketClient {
    param(
        [Parameter(Mandatory)]
        $Socket
    )

    $Socket.ReceiveTimeout = 1000
    $Socket.Client.NoDelay = $true
    $stream = $Socket.GetStream()
    return @{
        Reader = [System.IO.BinaryReader]::new($stream)
        Writer = [System.IO.BinaryWriter]::new($stream)
    }
}

❸ function Read-TlsRecordToken {
    param(
        [Parameter(Mandatory)]
        $Client
    )
    $reader = $Client.Reader
    $header = $reader.ReadBytes(5)
    $length = ([int]$header[3] -shl 8) -bor ($header[4])
    $data = @()
    ❹ while($length -gt 0) {
        $next = $reader.ReadBytes($length)
        if ($next.Length -eq 0) {
            throw "End of stream."
        }
        $data += $next
        $length -= $next.Length
    }

    Get-LsaAuthToken -Token ($header+$data)
}

```

```

}

❶ Use-NtObject($socket = [System.Net.Sockets.TcpClient]::new($Hostname, 443)) {
    $tcp_client = Get-SocketClient $socket

    ❷ $credout = New-LsaCredentialHandle -Package "Schannel" -UseFlag Outbound
    $client = New-LsaClientContext -CredHandle $credout -Target $Hostname
    -RequestAttribute ManualCredValidation

    ❸ while(!(Test-LsaContext -Context $client)) {
        ❹ if ($client.Token.Length -gt 0) {
            $tcp_client.Writer.Write($client.Token.ToArray())
        }

        ❺ $record = Read-TlsRecordToken -Client $tcp_client
        Update-LsaClientContext -Client $client -Token $record
    }

    ❻ $client.RemoteCertificate
}

```

Listing 15-30: A script for reading a TLS server certificate

We first define a couple of parameters, for the hostname of the server and the optional TCP port ❶. HTTPS uses the well-known port 443; however, TLS is not restricted to only that port, so you can change it if you want to target a different service.

We then define a couple of functions. The first one, `Get-SocketClient`, converts a TCP client object to a `BinaryReader` and `BinaryWriter` ❷. The TLS protocol has a relatively simple binary record structure, so using these classes makes it easier to parse the network traffic.

The second function, `Read-TlsRecordToken`, reads a single TLS record from the server and returns it as an authentication token ❸. We first read the 5-byte header from the record and extract the data's length, then we read the data from the stream. Because TCP is a streaming protocol, there is no guarantee that all the required data will be returned in a single read, so you'll have to perform the read in a loop until you've received everything you need ❹.

We now enter the body of the script. We start by making a TCP connection to the hostname and TCP port provided as arguments to the script ❺. We then convert the socket to the reader and writer objects. Next, we create the `Schannel` credentials and client context ❻, setting the client context tar-

get to the hostname and enabling manual credential validation, as we don't really care if the server certificate is invalid for the purposes of this example.

We can now loop until the client context has completed authentication ⑦. If there is a token to send to the server, we convert it to bytes and write it to the TCP socket ⑧. As we saw earlier, the TLS client and server can generate more than one TLS record, which the context must handle before generating a new token.

Once we've sent the client authentication token, we can read the next TLS record from the server and update the client ⑨. This loop will carry on until either the authentication completes successfully or an exception stops the script. Finally, we can return the server's certificate from the script ⑩.

Listing 15-31 shows how to use the script we wrote.

```
PS> $cert = .\get_server_cert.ps1 -Hostname www.microsoft.com
PS> $cert
Thumbprint                               Subject
-----
9B2B8AE65169AA477C5783D6480F296EF48CF14D  CN=www.microsoft.com,...

PS> $cert | Export-Certificate -FilePath output.cer
Directory: C:\demo
Mode                LastWriteTime         Length Name
----                -
-a-----          02-21  17:10             2173  output.cer
```

Listing 15-31: Getting the server certificate for www.microsoft.com and exporting it to a file

You call the script by providing the hostname of the server. Optionally, you could specify the TCP port, but in this case, we use port 443, better known as HTTPS, which is the script's default. The returned certificate is an object you can inspect using PowerShell. You can also export the certificate to a file using the `Export-Certificate` command.

Wrapping Up

This chapter began by describing security buffers and how they're used to pass information back and forth with the SSPI APIs during network authentication and the encryption and signing processes. It then provided

an overview of the Negotiate authentication protocol, which allows network authentication to take place when both parties aren't sure ahead of time what authentication protocol to use.

Next, we looked at some less commonly used security packages, secure channel and CredSSP. These have specific niches but also more complex usage compared to NTLM or Kerberos. We also discussed anonymous and identity network authentication in NTLM and Kerberos and covered network authentication inside a lowbox token sandbox (and I described how I circumvented this authentication multiple times).

The chapter finished with an overview of the security audit events generated when a user authenticates. You learned about the different event types used to describe whether a user's authentication succeeded or failed, and saw how to use these to figure out which users have attempted to authenticate to a workstation.

Final Thoughts

As we wrap up this final chapter, I hope you'll apply the information you've learned here about the internals of Windows security to your own endeavors. I've covered many areas in detail, ranging from the Security Reference Monitor and tokens to access checking and authentication, providing examples to demonstrate important topics.

However, I wasn't able to provide scripts to demonstrate every permutation of the features we discussed. For that reason, I recommend checking the help feature for the various commands provided with the `NtObjectManager` module and experimenting with their use. If you perform tests against a Windows virtual machine, there is little you can damage. (In fact, if your system develops a blue screen of death while you're experimenting, it might be a good idea to dig into why, as you might have found a security vulnerability.)

Following this chapter are some additional reference materials:

Appendix A contains a walkthrough for setting up a domain network for testing, and **Appendix B** contains a list of SDDL aliases.